Task 3 - Bank Marketing Decision Tree Classification

Introduction

This project focuses on building a Decision Tree Classifier to predict whether a customer will subscribe to a term deposit based on demographic and behavioral information collected from a bank marketing campaign.

The project involves:

- data exploration
- exploratory data analysis (EDA)
- preprocessing and encoding
- model building
- model evaluation
- feature importance analysis

The objective of this analysis is to understand customer behavior patterns and identify the key factors influencing subscription decisions. Various visualizations and machine learning techniques were used to interpret the dataset and evaluate model performance.

Dataset Source

Dataset: Bank Marketing Dataset Source: UCI Machine Learning Repository

The dataset contains customer demographic details, financial information, and campaign interaction history collected during direct marketing campaigns conducted by a Portuguese banking institution.

Project Objectives

- Understand the structure and characteristics of the dataset
- Perform exploratory data analysis to identify customer behavior patterns
- Preprocess and encode categorical variables for machine learning
- Build a Decision Tree Classification model
- Evaluate model performance using classification metrics
- Identify the most influential features affecting customer subscription decisions

Workflow

1. Import Libraries
2. Load Dataset

3. Basic Data Exploration
4. Exploratory Data Analysis
5. Data Preprocessing
6. Feature Encoding
7. Train-Test Split
8. Model Building
9. Model Evaluation
10. Confusion Matrix Visualization
11. Decision Tree Visualization
12. Feature Importance Analysis
13. Conclusion

Import Libraries

In [3]:

```
import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import matplotlib.ticker as ticker
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix
)

print("Libraries Imported")
```

```
Libraries Imported
```

Load Dataset

In [4]:

```
df = pd.read_csv("D:/Tanushree/prodigy/Task 3/bank/bank-full.csv", sep=';')

df.head()
```

Out[4]:

	age	job	marital	education	default	balance	housing	loan	contact	day	month	durati
0	58	management	married	tertiary	no	2143	yes	no	unknown	5	may	2
1	44	technician	single	secondary	no	29	yes	no	unknown	5	may	1
2	33	entrepreneur	married	secondary	no	2	yes	yes	unknown	5	may	
3	47	blue-collar	married	unknown	no	1506	yes	no	unknown	5	may	

	age	job	marital	education	default	balance	housing	loan	contact	day	month	durati
4	33	unknown	single	unknown	no	1	no	no	unknown	5	may	1

Basic EDA

In [5]:

```
df.shape
```

Out[5]:

```
(45211, 17)
```

In [6]:

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 45211 entries, 0 to 45210
Data columns (total 17 columns):
 #   Column     Non-Null Count  Dtype 
---  ------     --------------  ----- 
 0   age        45211 non-null  int64 
 1   job        45211 non-null  object
 2   marital    45211 non-null  object
 3   education  45211 non-null  object
 4   default    45211 non-null  object
 5   balance    45211 non-null  int64 
 6   housing    45211 non-null  object
 7   loan       45211 non-null  object
 8   contact    45211 non-null  object
 9   day        45211 non-null  int64 
 10  month      45211 non-null  object
 11  duration   45211 non-null  int64 
 12  campaign   45211 non-null  int64 
 13  pdays      45211 non-null  int64 
 14  previous   45211 non-null  int64 
 15  poutcome   45211 non-null  object
 16  y          45211 non-null  object
dtypes: int64(7), object(10)
memory usage: 5.9+ MB
```

In [7]:

```
# # Missing Values

df.isnull().sum()
```

Out[7]:

```
age          0
job          0
marital      0
education    0
default      0
balance      0
housing      0
loan         0
contact      0
day          0
month        0
duration     0
```

```
campaign     0
pdays        0
previous     0
poutcome     0
y            0
dtype: int64
```

In [8]:

```
# Duplicate

df.duplicated().sum()
```

Out[8]:

```
0
```

In [9]:

```
# Target Distribution - Count Values

df['y'].value_counts()
```

Out[9]:

```
y
no     39922
yes     5289
Name: count, dtype: int64
```

In [10]:

```
# Target Distribution - Percentage Distribution

df['y'].value_counts(normalize=True) * 100
```

Out[10]:

```
y
no     88.30152
yes    11.69848
Name: proportion, dtype: float64
```

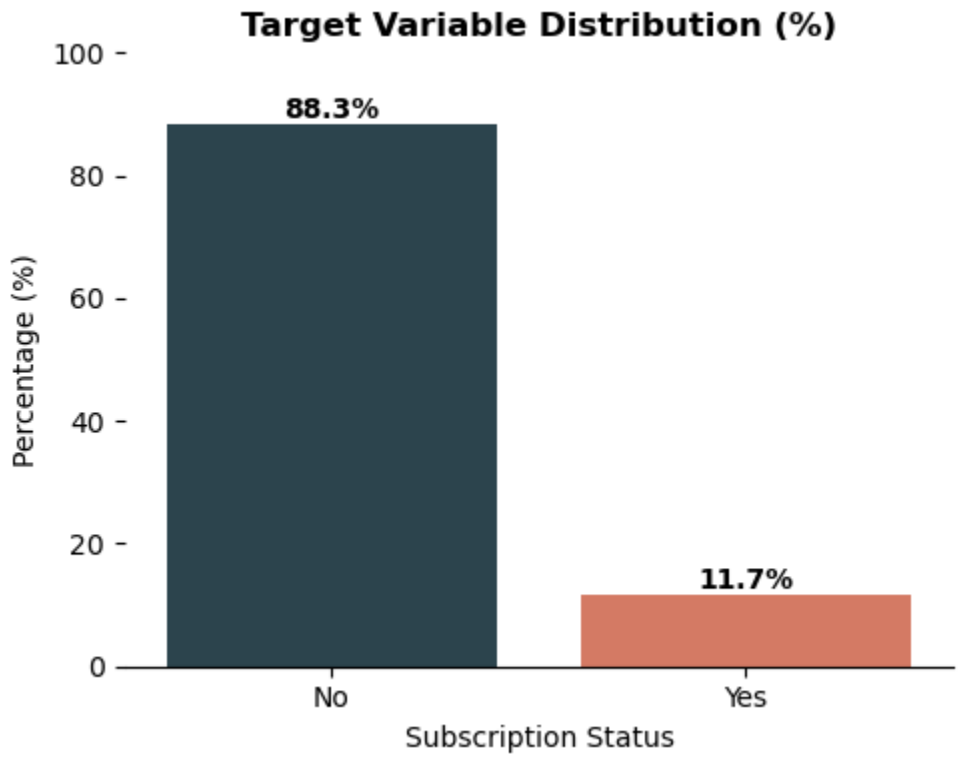
Exploratory Data Analysis

Target Variable Analysis

In [11]:

```
# Calculate target percentages
target_rate = df['y'].value_counts(normalize=True) * 100

# Plot
plt.figure(figsize=(5,4))

ax = sns.barplot(
    x=target_rate.index,
    y=target_rate.values,
    hue=target_rate.index,
    palette={
        'no': '#264653',
        'yes': '#E76F51'
    },
    legend=False
)
```

```
# Add percentage labels
for p in ax.patches:

    height = p.get_height()

    ax.annotate(
        f'{height:.1f}%',
        (
            p.get_x() + p.get_width()/2,
            height
        ),
        ha='center',
        va='bottom',
        fontsize=10,
        fontweight='bold'
    )

sns.despine(top=True, right=True, left=True)

plt.title(
    'Target Variable Distribution (%)',
    fontsize=12,
    fontweight='bold'
)

plt.xlabel('Subscription Status')
plt.ylabel('Percentage (%)')

plt.xticks([0,1], ['No', 'Yes'])

plt.ylim(0, 100)

plt.tight_layout()
plt.show()
```



Demographic vs Target

In [12]:

```
age_bins = [10, 20, 30, 40, 50, 60, 70, 80, 90, 100]

age_labels = [
    '11-20',
    '21-30',
    '31-40',
    '41-50',
    '51-60',
    '61-70',
    '71-80',
    '81-90',
    '91-100'
]

df['Age_Group'] = pd.cut(
    df['age'],
    bins=age_bins,
    labels=age_labels,
    right=True
)

print(df["Age_Group"].head())
```

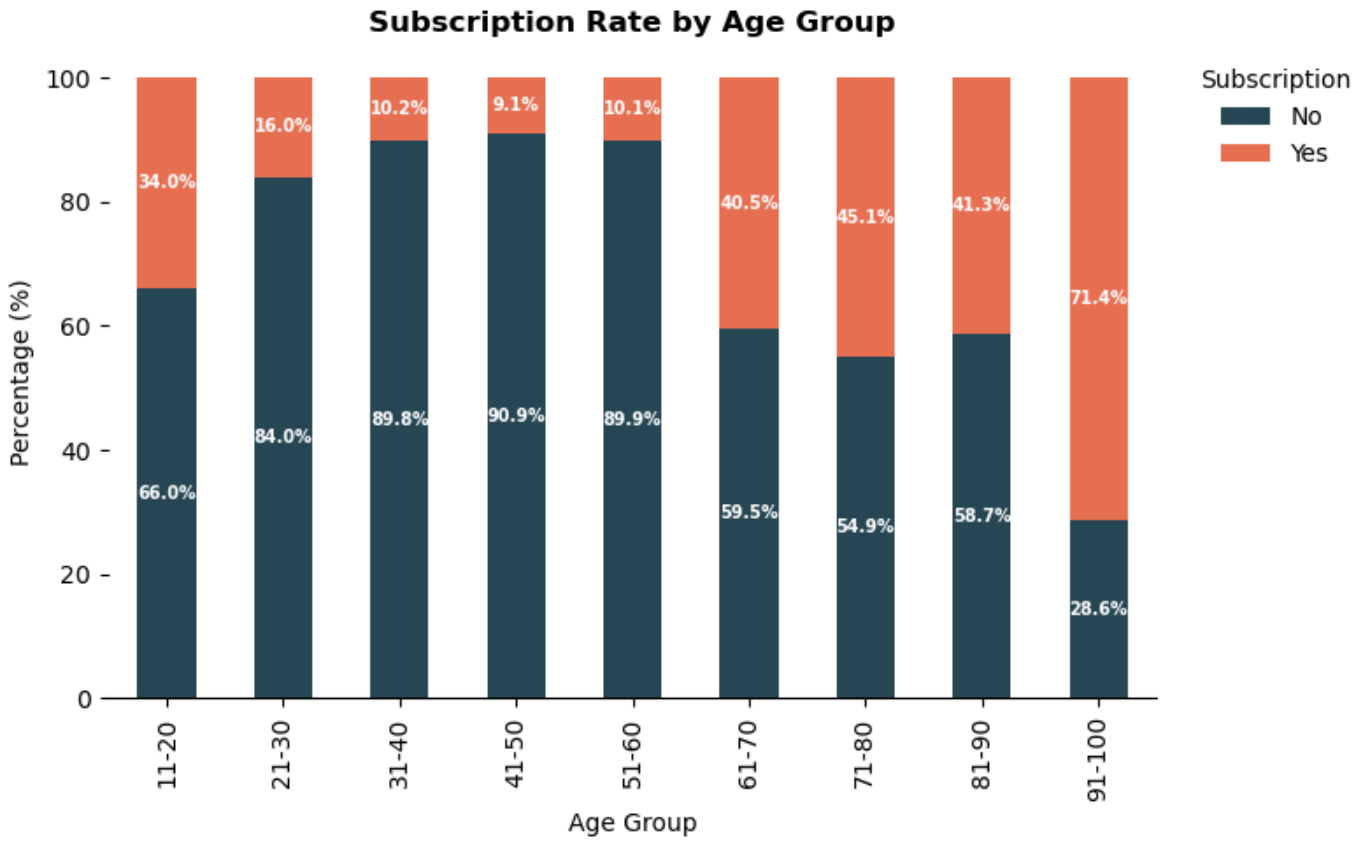
```
0    51-60
1    41-50
2    31-40
3    41-50
4    31-40
Name: Age_Group, dtype: category
Categories (9, object): ['11-20' < '21-30' < '31-40' < '41-50' ... '61-70' < '71-80' <
'81-90' < '91-100']
```

Plot: Age Distribution

In [13]:

```
# Rate% distribution by Age Group

age_subscription = pd.crosstab(
    df['Age_Group'],
    df['y'],
    normalize='index'
) * 100

# Plot
ax = age_subscription.plot(
    kind='bar',
    stacked=True,
    figsize=(8,5),
    color=['#264653', '#E76F51']
)

# Add percentage labels
for container in ax.containers:
```

```
    labels = [
        f'{v.get_height():.1f}%'
        if v.get_height() > 0 else ''
        for v in container
    ]

    ax.bar_label(
        container,
        labels=labels,
        label_type='center',
        fontsize=7,
        color='white',
        fontweight='bold'
    )

sns.despine(top=True, right=True, left=True)

plt.title(
    'Subscription Rate by Age Group',
    fontsize=12,
    fontweight='bold'
)

plt.xlabel('Age Group')
plt.ylabel('Percentage (%)')

plt.legend(
    title='Subscription',
    labels=['No', 'Yes'],
    frameon=False,
    bbox_to_anchor=(1.02, 1),
    loc='upper left'
)

plt.tight_layout()
plt.show()
```



Plot: Job vs. Subscription

In [14]:

```
df['job'].unique()
```

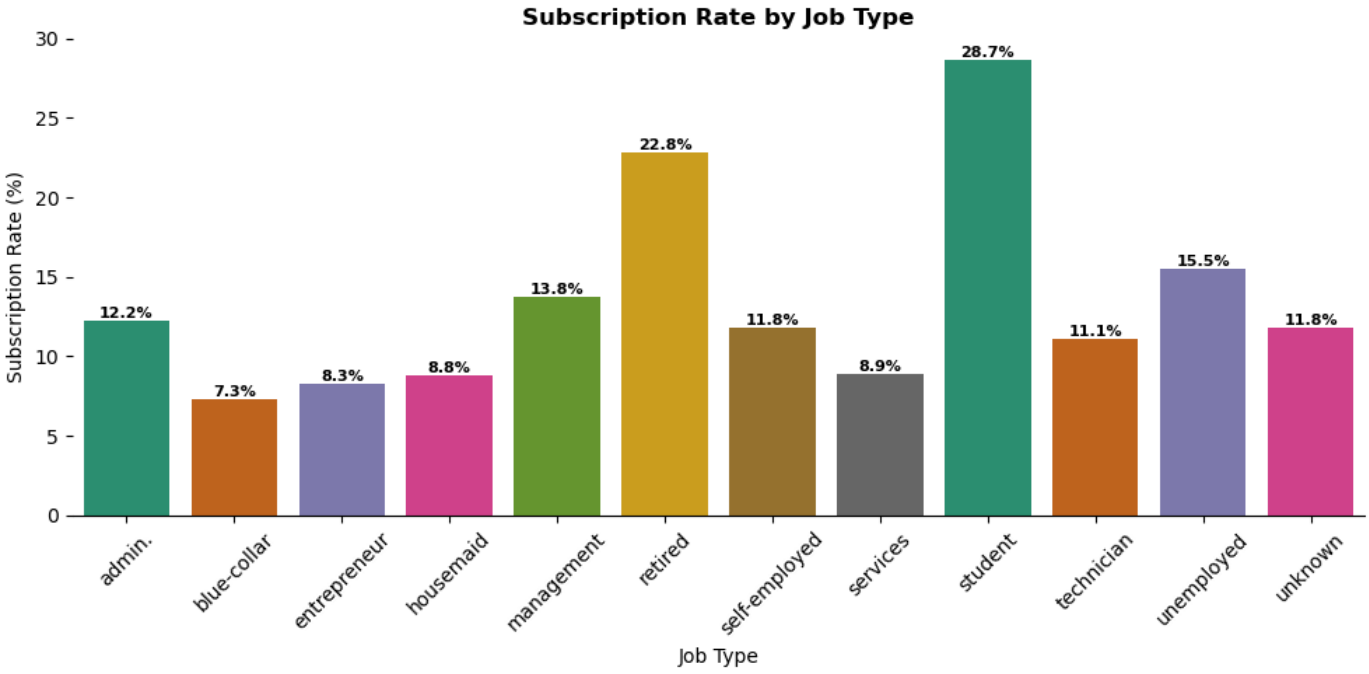
Out[14]:

```
array(['management', 'technician', 'entrepreneur', 'blue-collar',
       'unknown', 'retired', 'admin.', 'services', 'self-employed',
       'unemployed', 'housemaid', 'student'], dtype=object)
```

In [15]:

```
# Calculate Target percentages
Job_sub = pd.crosstab(
    df['job'],
    df['y'],
    normalize='index'
) * 100

# Convert index into column
Job_sub = Job_sub.reset_index()

# PLot
plt.figure(figsize=(10, 5))

ax = sns.barplot(
    data=Job_sub,
    x='job',
    y='yes',
    hue='job',
    palette='Dark2',
    legend=False
)
```

```
# Add percentage labels
for p in ax.patches:

    height = p.get_height()

    ax.annotate(
        f'{height:.1f}%',
        (
            p.get_x() + p.get_width()/2,
            height
        ),
        ha='center',
        va='bottom',
        fontsize=8,
        fontweight='bold'
    )

sns.despine(top=True, right=True, left=True)

plt.title(
    'Subscription Rate by Job Type',
    fontsize=12,
    fontweight='bold'
)

plt.xlabel('Job Type')
plt.ylabel('Subscription Rate (%)')

plt.xticks(rotation=45)

plt.tight_layout()
plt.show()
```



Plot: Martial Status

In [16]:

```
df['marital'].unique()
```

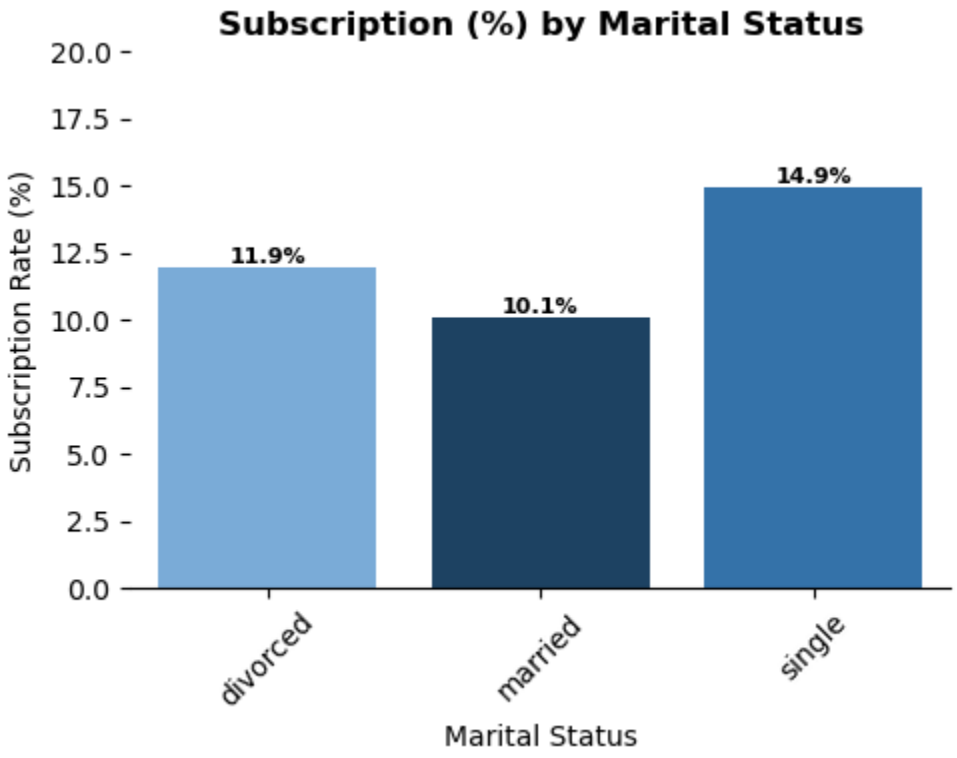
Out[16]:

```
array(['married', 'single', 'divorced'], dtype=object)
```

In [17]:

```
# Subscrition Rate (%)
Marital_Subs = pd.crosstab(
    df['marital'],
    df['y'],
    normalize='index'
) * 100

# Convert index into columns
Marital_Subs = Marital_Subs.reset_index()

# Plot
plt.figure(figsize=(5, 4))

ax = sns.barplot(
    data=Marital_Subs,
    x='marital',
    y='yes',
    hue='marital',
    palette={
        'married': '#12436D',
        'single': '#2073BC',
        'divorced': '#6BACE6'
    },
    legend=False
)

# Add labels
for p in ax.patches:

    height = p.get_height()

    ax.annotate(
        f'{height:.1f}%',
        (
            p.get_x() + p.get_width()/2,
            height
        ),
        ha='center',
        va='bottom',
        fontsize=8,
        fontweight='bold'
    )

sns.despine(top=True, right=True, left=True)

plt.title(
    'Subscription (%) by Marital Status',
    fontsize=12,
    fontweight='bold'
)

plt.xlabel('Marital Status')
```

```
plt.ylabel('Subscription Rate (%)')

plt.xticks(rotation=45)

plt.ylim(0, 20)
plt.tight_layout()
plt.show()
```



Plot: Educational status by Subsricption

In [18]:

```
df['education'].unique()
```

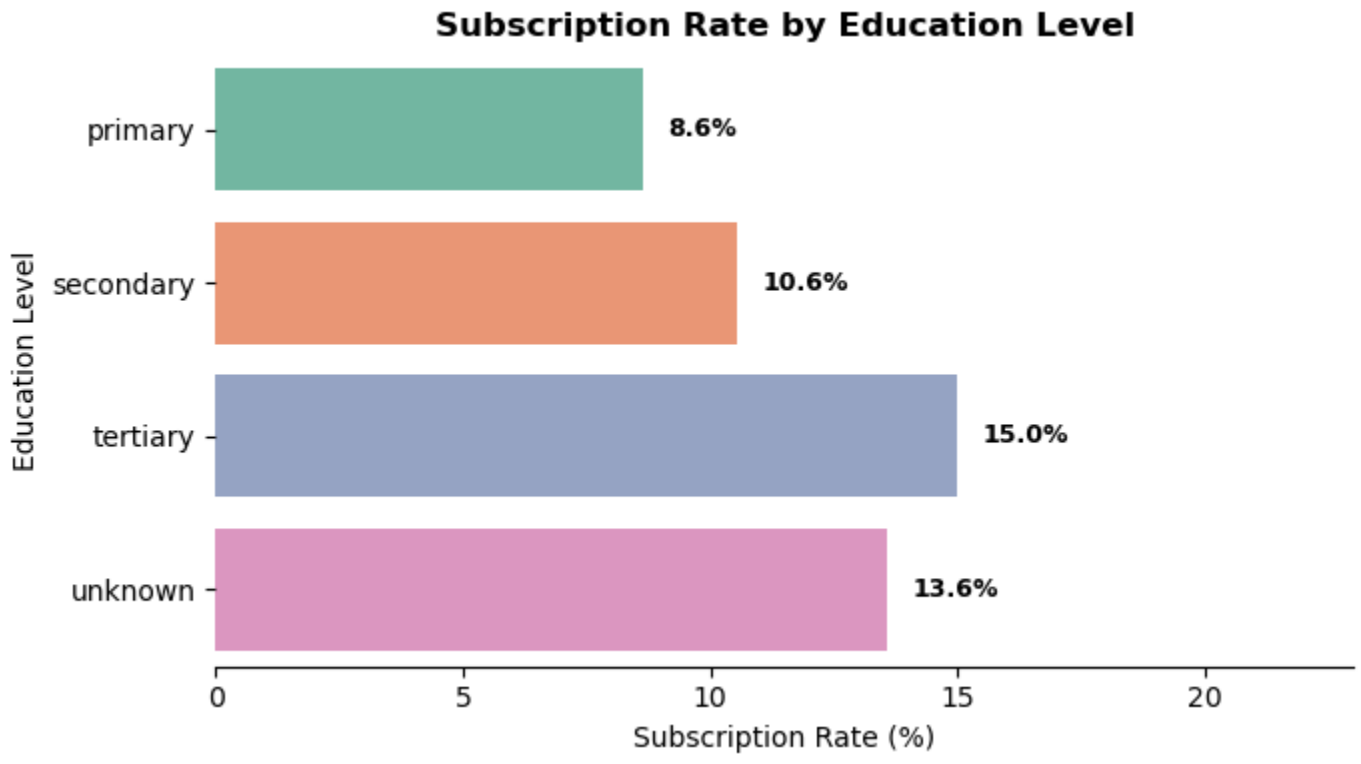
Out[18]:

```
array(['tertiary', 'secondary', 'unknown', 'primary'], dtype=object)
```

In [19]:

```
# Subscription Rate (%)
Edu_Subs = pd.crosstab(
    df['education'],
    df['y'],
    normalize='index'
) * 100

Edu_Subs = Edu_Subs.reset_index()

# Plot
plt.figure(figsize=(7,4))

ax = sns.barplot(
    data=Edu_Subs,
    y='education',
    x='yes',
    hue='education',
    palette='Set2',
```

```
    legend=False
)

# Add labels
for p in ax.patches:

    width = p.get_width()

    ax.annotate(
        f'{width:.1f}%',
        (
            width + 0.5,
            p.get_y() + p.get_height()/2
        ),
        ha='left',
        va='center',
        fontsize=9,
        fontweight='bold'
    )

sns.despine(top=True, right=True, left=True)

plt.title(
    'Subscription Rate by Education Level',
    fontsize=12,
    fontweight='bold'
)

plt.xlabel('Subscription Rate (%)')
plt.ylabel('Education Level')

plt.xlim(0, int(Edu_Subs['yes'].max()) + 8)

plt.tight_layout()
plt.show()
```

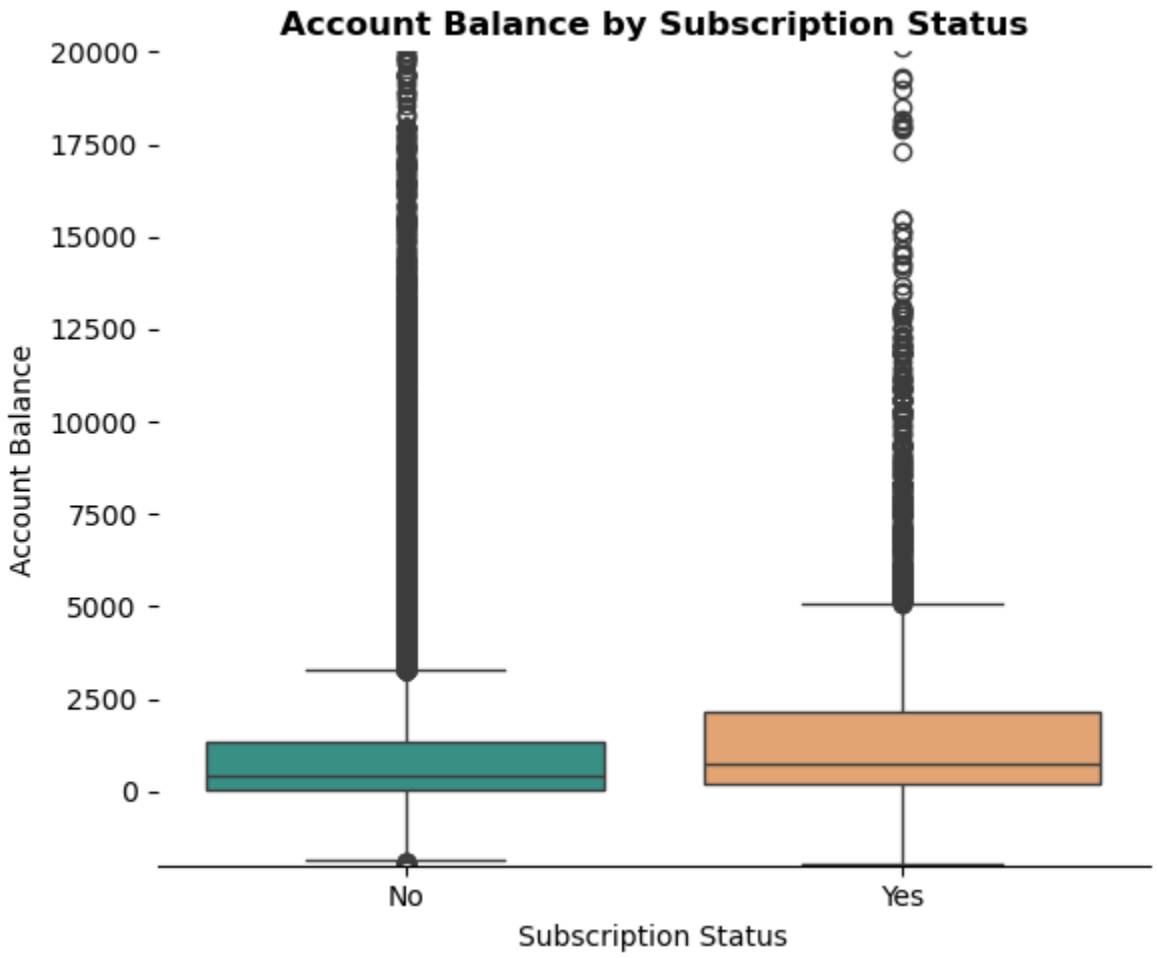


Financial vs. Target

Plot: Balance Distribution

In [20]:

```
plt.figure(figsize=(6,5))

ax = sns.boxplot(
    data=df,
    x='y',
    y='balance',
    hue='y',
    palette={
        'no': '#2A9D8F',
        'yes': '#F4A261'
    }
)

sns.despine(top=True, right=True, left=True)

plt.title(
    'Account Balance by Subscription Status',
    fontsize=12,
    fontweight='bold'
)

plt.xlabel('Subscription Status')
plt.ylabel('Account Balance')

plt.xticks([0,1], ['No', 'Yes'])

plt.ylim(-2000, 20000)

plt.tight_layout()
plt.show()
```

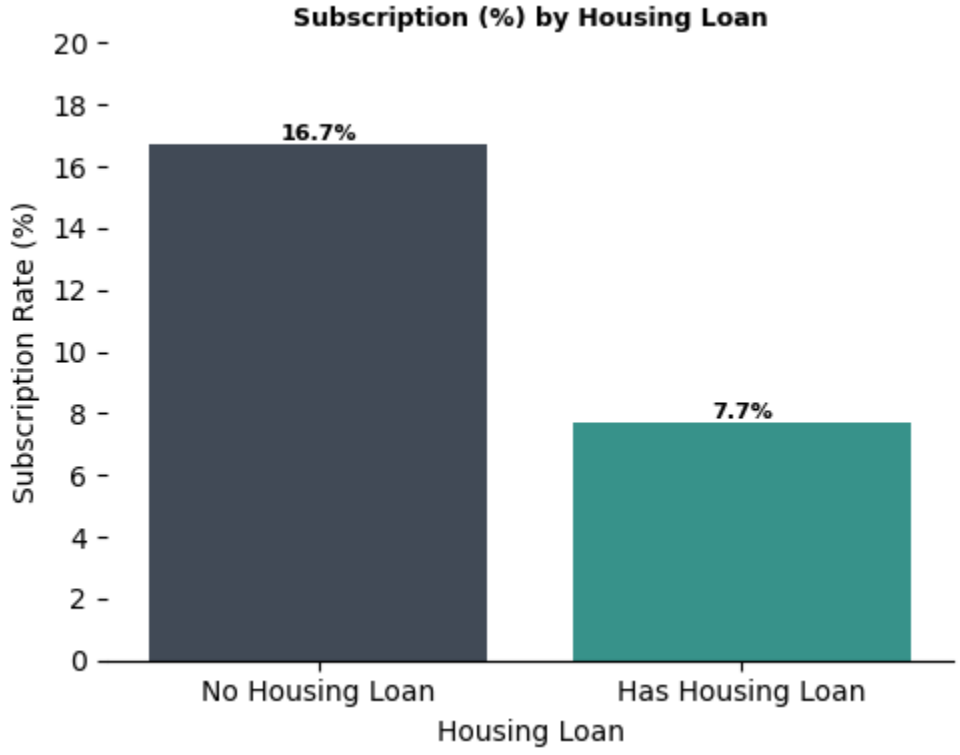


Plot: Housing Loan vs. Subscription

In [21]:

```
# Subscrition Rate (%)
Hous_Subs = pd.crosstab(
    df['housing'],
    df['y'],
    normalize='index'
) * 100

# Convert index into columns
Hous_Subs = Hous_Subs.reset_index()

# Plot
plt.figure(figsize=(5, 4))

ax = sns.barplot(
    data=Hous_Subs,
    x='housing',
    y='yes',
    hue='housing',
    palette=['#3D4A5A', '#28A197']
)

# Add labels
for p in ax.patches:

    height = p.get_height()
```

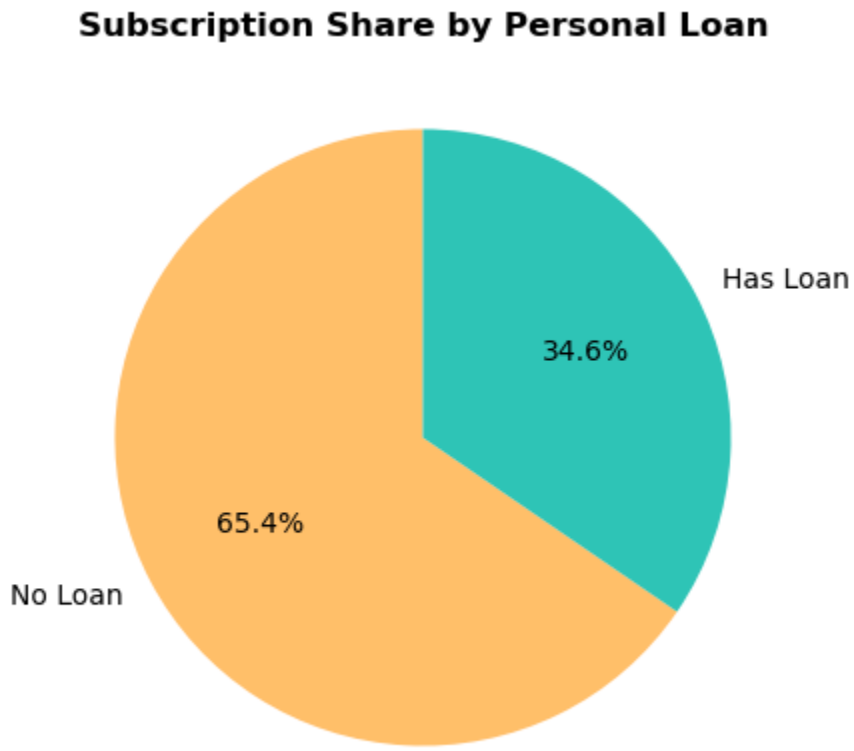
```
    ax.annotate(
        f'{height:.1f}%',
        (
            p.get_x() + p.get_width()/2,
            height
        ),
        ha='center',
        va='bottom',
        fontsize=8,
        fontweight='bold'
    )

sns.despine(top=True, right=True, left=True)

plt.title(
    'Subscription (%) by Housing Loan',
    fontsize=9,
    fontweight='bold'
)

plt.xlabel('Housing Loan')
plt.ylabel('Subscription Rate (%)')

plt.xticks([0,1], ['No Housing Loan', 'Has Housing Loan'])
ax.yaxis.set_major_locator(ticker.MaxNLocator(integer=True))

plt.ylim(0, 20)
plt.tight_layout()
plt.show()
```



Plot: Loan affecting Subscription Rate (%)

In [22]:

```
loan_rate = df.groupby('loan')['y'].apply(
    lambda x: (x == 'yes').mean() * 100
)

plt.figure(figsize=(5,5))

plt.pie(
    loan_rate,
    labels=['No Loan', 'Has Loan'],
    autopct='%1.1f%%',
    colors=['#FFBF69', '#2EC4B6'],
    startangle=90
)

plt.title(
    'Subscription Share by Personal Loan',
    fontsize=12,
    fontweight='bold'
)

plt.show()
```

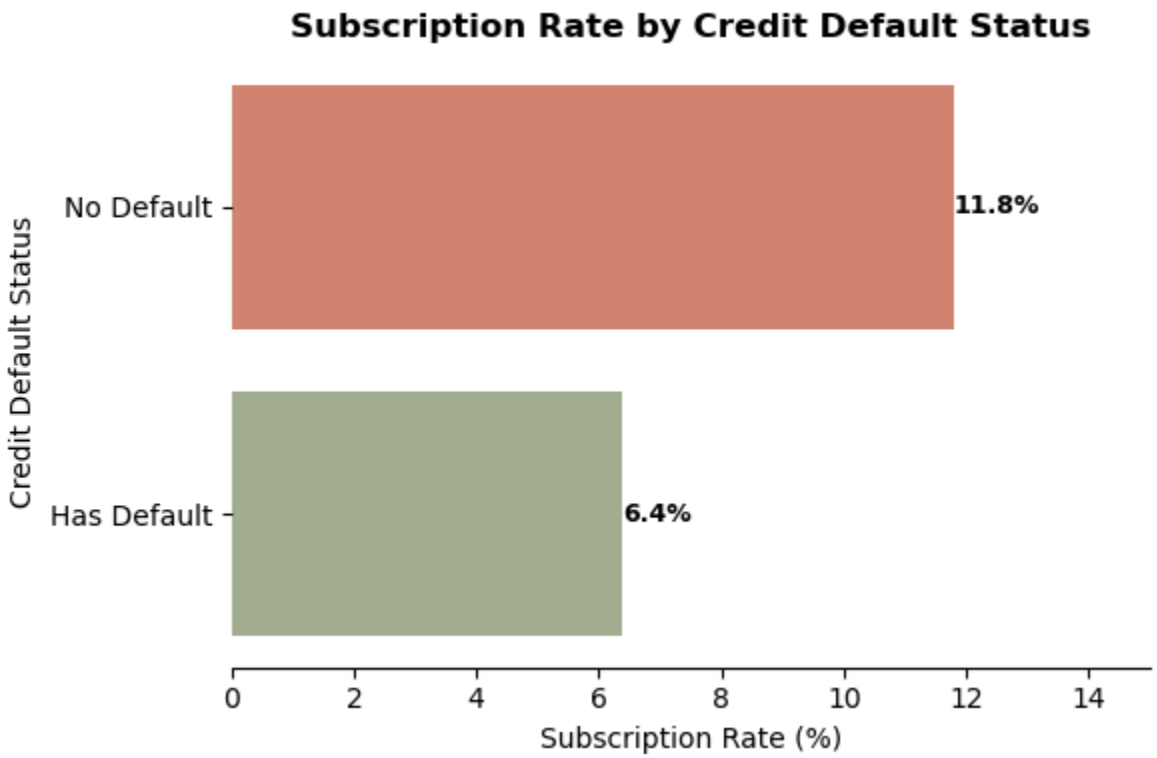


Plot: Credit Default Status

In [39]:

```
# Subscription Rate (%)
Default_Subs = pd.crosstab(
    df['default'],
    df['y'],
    normalize='index'
) * 100
```

```
# Convert index into columns
Default_Subs = Default_Subs.reset_index()

# Plot
plt.figure(figsize=(6,4))

ax = sns.barplot(
    data=Default_Subs,
    y='default',
    x='yes',
    hue='default',
    palette={
        'no': '#E07A5F',
        'yes': '#A3B18A'
    },
    legend=False
)

# Add labels
for p in ax.patches:

    width = p.get_width()

    ax.annotate(
        f'{width:.1f}%',
        (
            width,
            p.get_y() + p.get_height()/2
        ),
        ha='left',
        va='center',
        fontsize=9,
        fontweight='bold'
    )

sns.despine(top=True, right=True, left=True, bottom=False)
ax.xaxis.set_major_locator(ticker.MaxNLocator(integer=True))

plt.title(
    'Subscription Rate by Credit Default Status',
    fontsize=12,
    fontweight='bold'
)

plt.xlabel('Subscription Rate (%)')
plt.ylabel('Credit Default Status')

plt.yticks(
    [0,1],
    ['No Default', 'Has Default']
)

plt.xlim(0, 15)

plt.tight_layout()
plt.show()
```



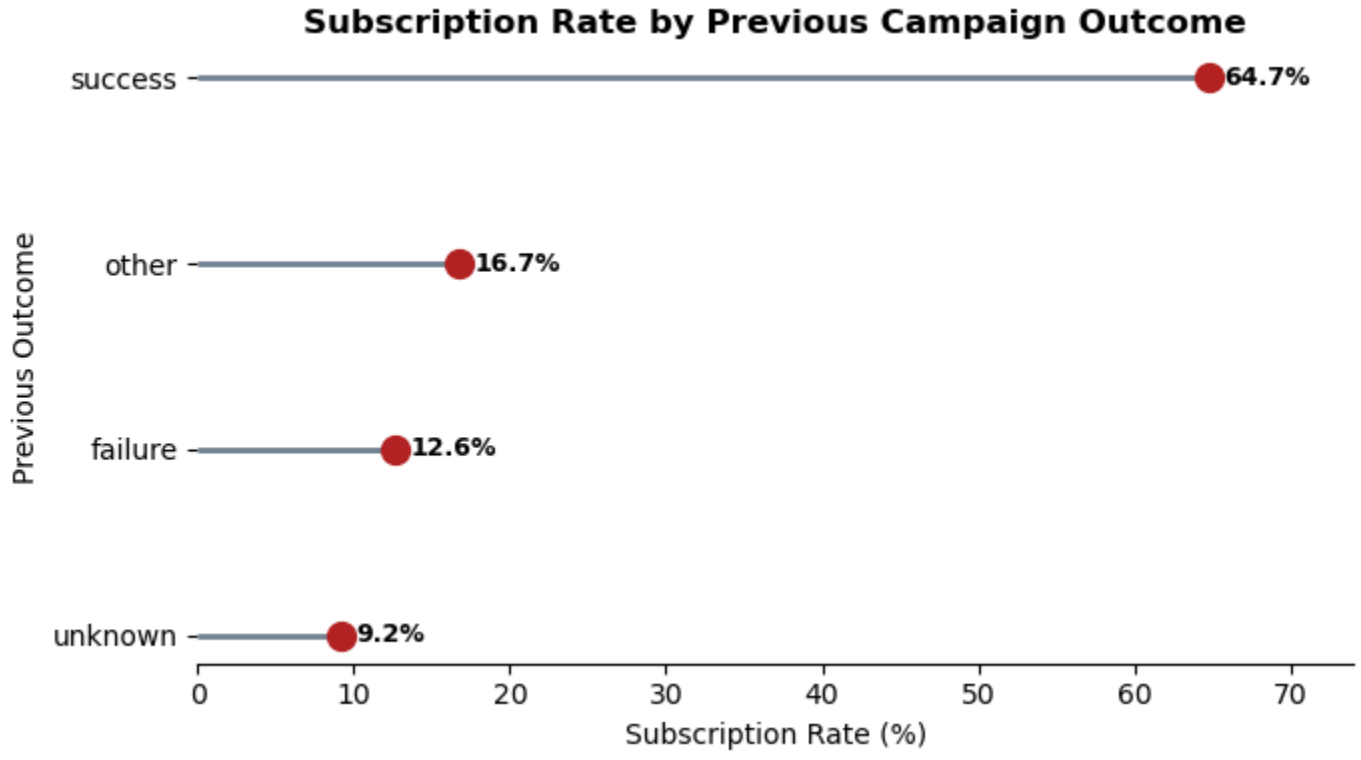
Campaign Behaviour vs Target

Plot: Previous Outcome & Subscription Rate

In [24]:

```
# Subscription Rate (%)
Prev_Subs = pd.crosstab(
    df['poutcome'],
    df['y'],
    normalize='index'
) * 100

Prev_Subs = Prev_Subs.reset_index()

# Sort values (optional but cleaner)
Prev_Subs = Prev_Subs.sort_values('yes')

# Plot
plt.figure(figsize=(7,4))

# Lines
plt.hlines(
    y=Prev_Subs['poutcome'],
    xmin=0,
    xmax=Prev_Subs['yes'],
    color='slategray',
    linewidth=2
)

# Dots
plt.plot(
    Prev_Subs['yes'],
    Prev_Subs['poutcome'],
```

```
    'o',
    color='firebrick',
    markersize=10
)

# Labels
for i, value in enumerate(Prev_Subs['yes']):

    plt.text(
        value + 1,
        i,
        f'{value:.1f}%',
        va='center',
        fontsize=9,
        fontweight='bold'
    )

sns.despine(top=True, right=True, left=True)

plt.title(
    'Subscription Rate by Previous Campaign Outcome',
    fontsize=12,
    fontweight='bold'
)

plt.xlabel('Subscription Rate (%)')
plt.ylabel('Previous Outcome')

plt.xlim(0, int(Prev_Subs['yes'].max()) + 10)

plt.tight_layout()
plt.show()
```

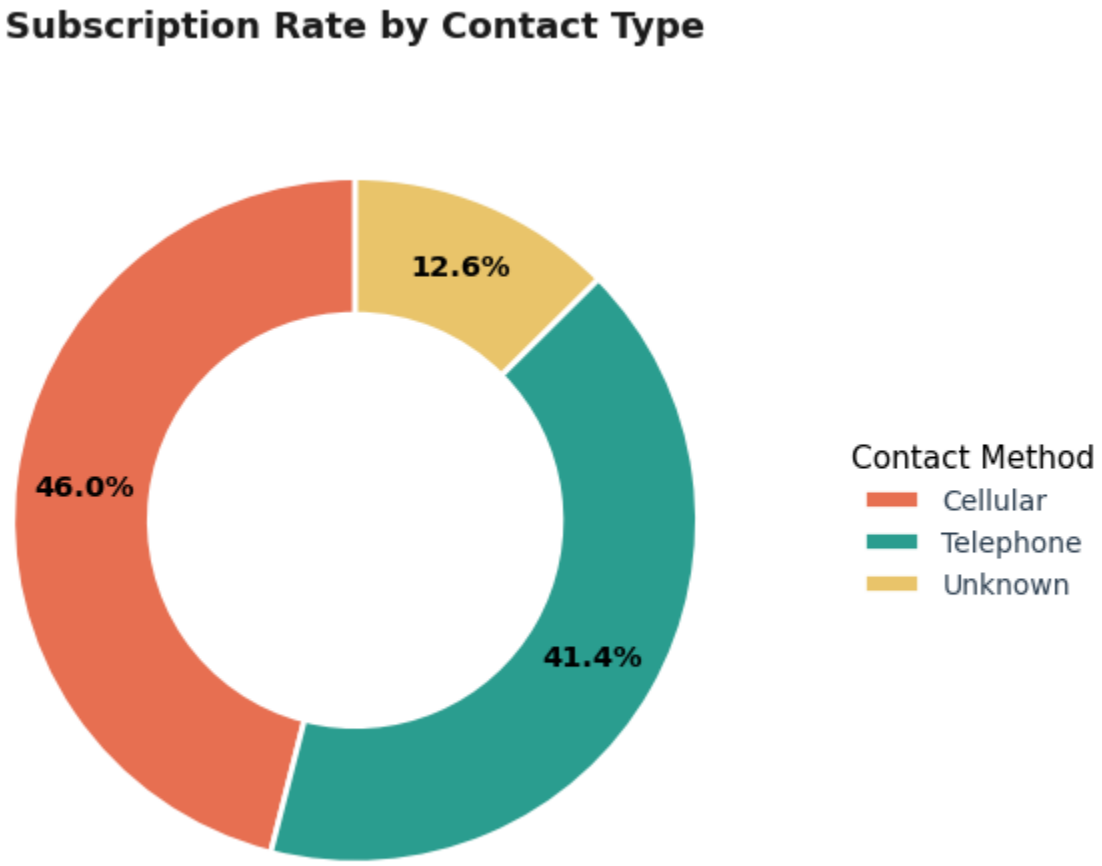


Plot: Contact Type vs. Subscription Rate

In [25]:

```
# --- Safety Check: Mock data structure if running independently ---
if 'df' not in locals():
    data = {
        'contact': ['cellular', 'telephone', 'unknown'],
        'yes': [15.0, 5.0, 4.0] # Sample percentage conversion rates
    }
    Contact_Subs = pd.DataFrame(data)
else:
    # Calculate Crosstab as done in your pipeline
    Contact_Subs = pd.crosstab(df['contact'], df['y'], normalize='index') * 100
    Contact_Subs = Contact_Subs.reset_index()

# Plot setup
plt.figure(figsize=(7, 5))

# NEW: Premium, distinct multi-color palette (replaces the blues)
custom_colors = ['#E76F51', '#2A9D8F', '#E9C46A']

# Create the donut wedges
wedges, texts, autotexts = plt.pie(
    Contact_Subs['yes'],
    autopct='%1.1f%%',
    startangle=90,
    colors=custom_colors,

    # Donut effect
    wedgeprops={
        'width': 0.4,
        'edgecolor': 'white',
        'linewidth': 2
    },

    # Adjust position of numbers to sit perfectly centered inside the ring
    pctdistance=0.8
)

# Style the numerical percentages inside the slices
plt.setp(
    autotexts,
    color='black',       # High-contrast crisp white text
    fontsize=10.5,
    fontweight='bold'
)

# Add Legend to the RIGHT CENTER with crisp charcoal font styling
plt.legend(
    wedges,
    Contact_Subs['contact'].str.title(), # Capitalizes first letter cleanly
    title='Contact Method',
    title_fontsize=11,
    fontsize=10,
    frameon=False,
    bbox_to_anchor=(1.05, 0.5), # Positions it cleanly to the right center
    loc='center left',
    labelcolor='#2C3E50'        # Sharp visibility font styling
)
```

```
# High-contrast near-black title
plt.title(
    'Subscription Rate by Contact Type',
    fontsize=13,
    fontweight='bold',
    pad=20,
    color='#1A1A1A'
)

plt.tight_layout()
plt.show()
```



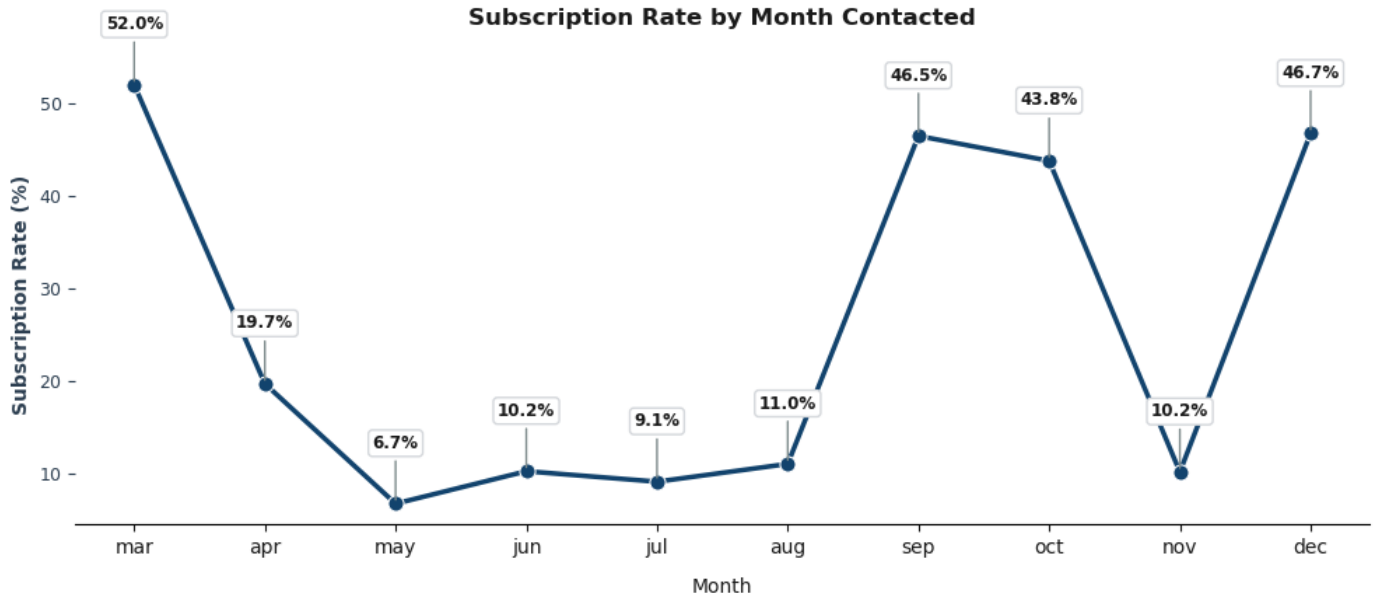
Plot: Month Contacted vs Subscription Rate

In [26]:

```
# Subscrition Rate (%)
MonCont_Subs = pd.crosstab(
    df['month'],
    df['y'],
    normalize='index'
) * 100

MonCont_Subs = MonCont_Subs.reset_index()

# Sort Months
month_order = ['mar', 'apr', 'may', 'jun', 'jul', 'aug', 'sep', 'oct', 'nov', 'dec']
MonCont_Subs['month'] = pd.Categorical(MonCont_Subs['month'], categories=month_order, or
MonCont_Subs = MonCont_Subs.sort_values('month').dropna().reset_index(drop=True)
```

```python
# Plot
plt.figure(figsize=(10 ,4.5))

sns.lineplot(
    data=MonCont_Subs,
    x='month',
    y='yes',
    marker='o',
    markersize=8,
    linewidth=2.5,
    color='#12436D',
    errorbar=None
)


# Labels
for i, value in enumerate(MonCont_Subs['yes']):

    plt.annotate(
        f'{value:.1f}%',

        # point location
        xy=(i, value),

        # label position above point
        xytext=(i, value + 6),

        ha='center',
        fontsize=8.5,
        fontweight='bold',
        color='#1A1A1A',

        # Arrow line
        arrowprops=dict(
            arrowstyle='-',
            color='#7F8C8D',
            lw=1
        ),

        # label background
        bbox=dict(
            boxstyle='round,pad=0.25',
            facecolor='white',
            edgecolor='#D5D8DC'
        )

    )

sns.despine(top=True, right=True, left=True)

plt.title('Subscription Rate by Month Contacted', fontsize=12, fontweight='bold', pad=20
plt.ylabel('Subscription Rate (%)', fontsize=10, color='#2C3E50', fontweight='bold')
plt.xlabel('Month', fontsize=10, labelpad=10, color='#1A1A1A')


plt.tick_params(axis='y', colors='#2C3E50', labelsize=9)
plt.tick_params(axis='x', colors='#1A1A1A', labelsize=10)
```

```
plt.tight_layout()
plt.show()
```

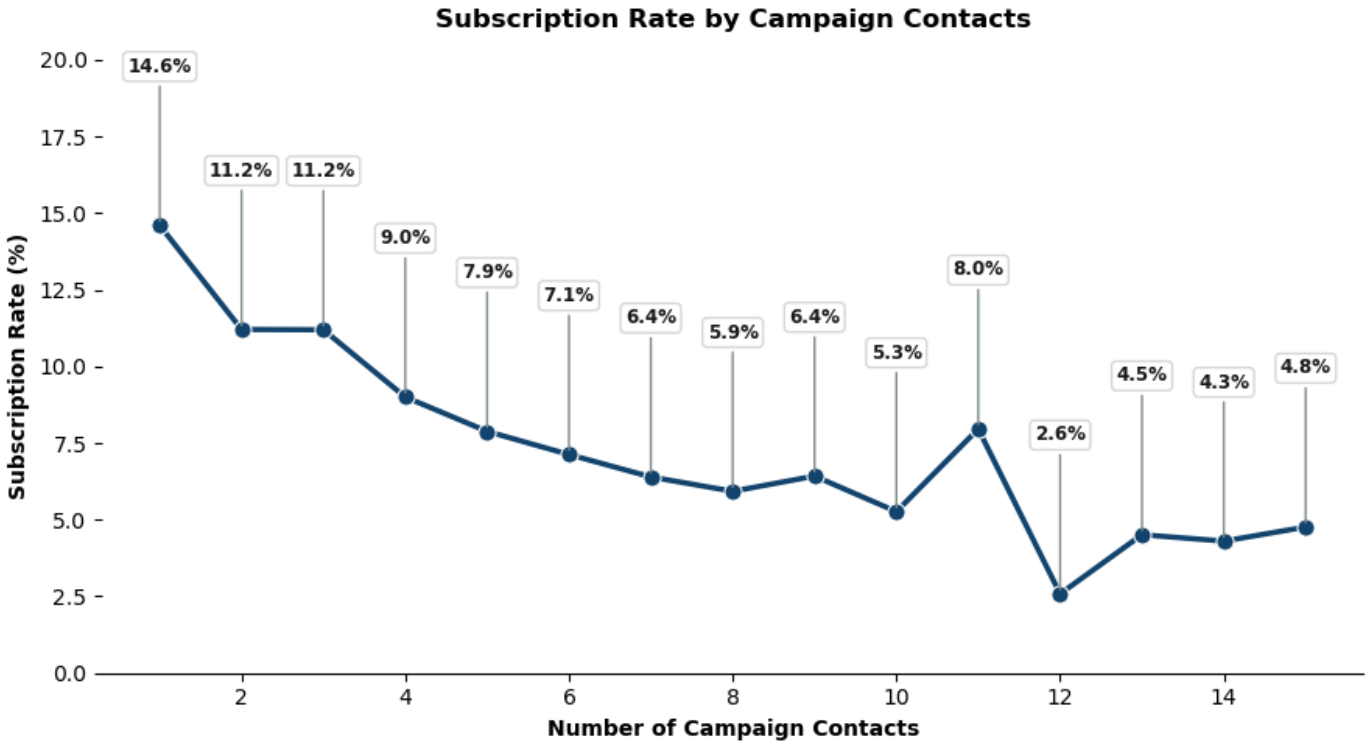


Plot: Campaign Contacts vs Subscription Rate

In [27]:

```
# Subscription Rate (%)
Campaign_Subs = pd.crosstab(
    df['campaign'],
    df['y'],
    normalize='index'
) * 100

Campaign_Subs = Campaign_Subs.reset_index()

# Keep meaningful campaign counts
Campaign_Subs = Campaign_Subs[Campaign_Subs['campaign'] <= 15]

# Plot
plt.figure(figsize=(9,5))

sns.lineplot(
    data=Campaign_Subs,
    x='campaign',
    y='yes',
    marker='o',
    markersize=8,
    linewidth=2.5,
    color='#12436D'
)

# Labels with pointer lines
for i, row in Campaign_Subs.iterrows():

    plt.annotate(
        f"{row['yes']:.1f}%",

        # point location
        xy=(row['campaign'], row['yes']),
```

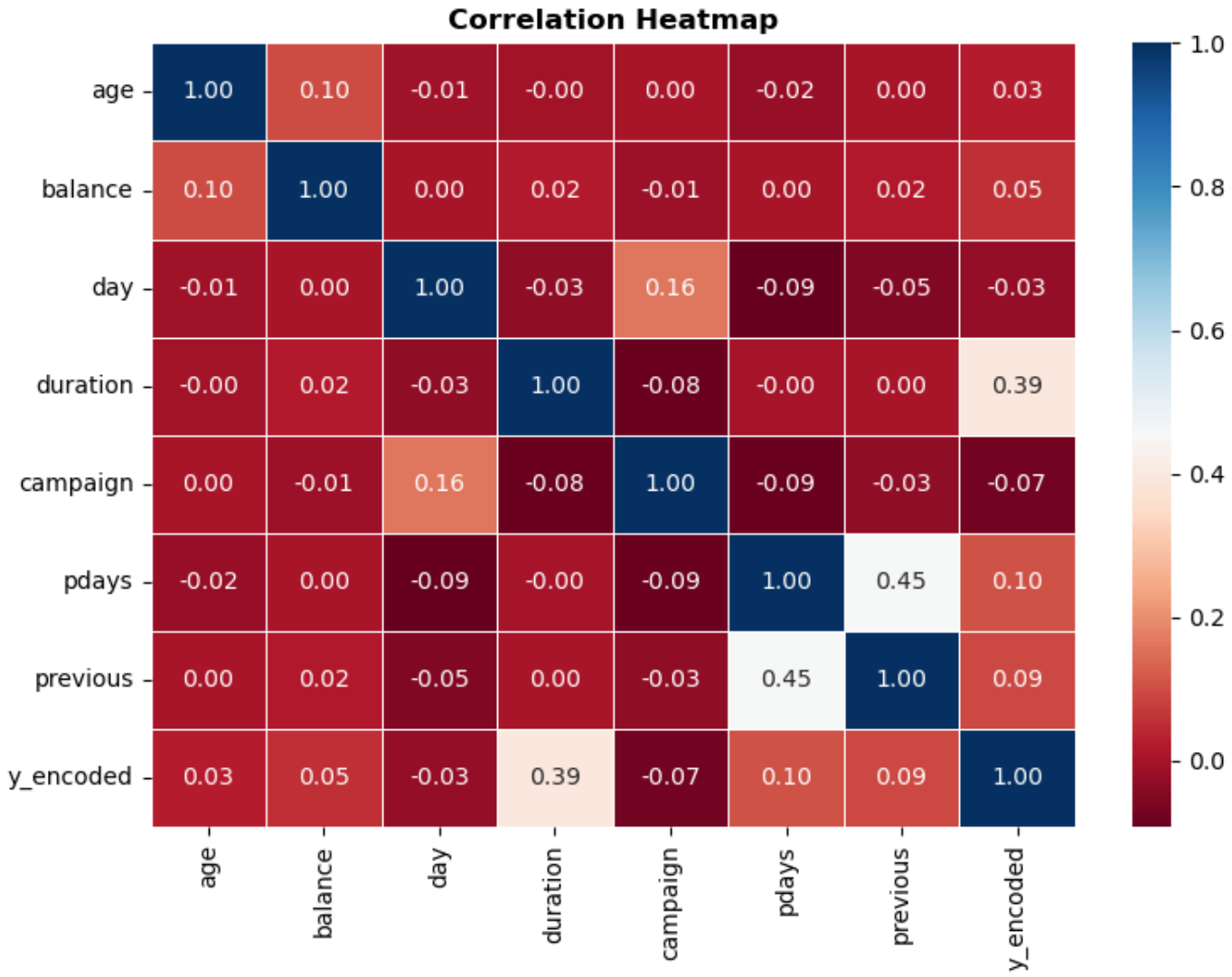
```
        # label position
        xytext=(row['campaign'], row['yes'] + 5),

        ha='center',
        fontsize=8.5,
        fontweight='bold',
        color='#1A1A1A',

        # pointer line
        arrowprops=dict(
            arrowstyle='-',
            color='#7F8C8D',
            lw=1
        ),

        # label box
        bbox=dict(
            boxstyle='round,pad=0.25',
            facecolor='white',
            edgecolor='#D5D8DC'
        )
    )

sns.despine(top=True, right=True, left=True)

plt.title(
    'Subscription Rate by Campaign Contacts',
    fontsize=12,
    fontweight='bold',
    pad=15
)

plt.xlabel(
    'Number of Campaign Contacts',
    fontsize=10,
    fontweight='bold'
)

plt.ylabel(
    'Subscription Rate (%)',
    fontsize=10,
    fontweight='bold'
)

plt.ylim(0, 20)

plt.tight_layout()
plt.show()
```



Correlation Heatmap

In [28]:

```
# Encode target
df['y_encoded'] = df['y'].map({'no': 0, 'yes': 1})

# Numeric columns
numeric_df = df.select_dtypes(include=['int64', 'float64'])

# Correlation
corr = numeric_df.corr()

# Plot
plt.figure(figsize=(8,6))

sns.heatmap(
    corr,
    annot=True,
    cmap='RdBu',
    fmt='.2f',
    linewidths=0.5
)

plt.title(
    'Correlation Heatmap',
    fontsize=12,
    fontweight='bold'
)

plt.tight_layout()
plt.show()
```



Data Preprocessing

In [29]:

```
# Step 1: Separate Features & Target
X = df.drop(
    ['y', 'y_encoded'],
    axis=1
)

# Target
y = df['y_encoded']
```

In [30]:

```
# Step 3: Encode Categorical Features

X = pd.get_dummies(
    X,
    drop_first=True
)
```

In [31]:

```
# Step 4: Train-Test Split
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

In [32]:

```
# Verify Shapes

print(X_train.shape)
print(X_test.shape)
print(y_train.shape)
print(y_test.shape)
```

```
(36168, 50)
(9043, 50)
(36168,)
(9043,)
```

Model Building

In [33]:

```
# Create Model
dt_model = DecisionTreeClassifier(
    max_depth=4,
    random_state=42
)

# Train Model
dt_model.fit(X_train, y_train)

# Predictions
y_pred = dt_model.predict(X_test)
```

Model Evaluation

In [34]:

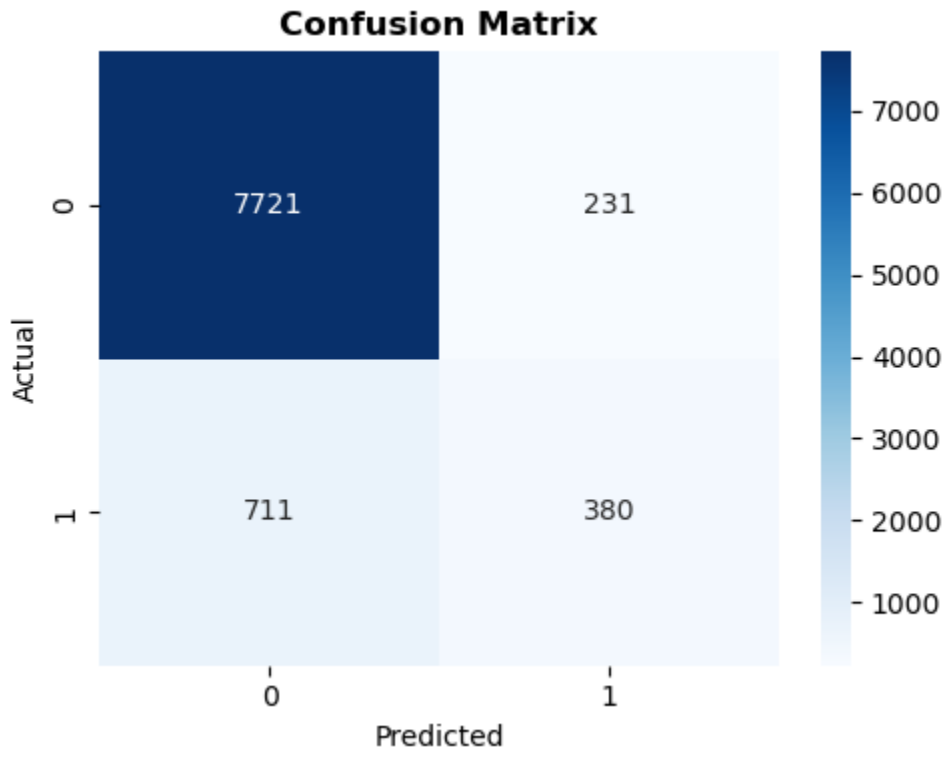
```
# Accuracy Store

accuracy = accuracy_score(y_test, y_pred)

print(f"The Decision Tree model achieved an accuracy of {accuracy:.2%}.")

# Classification Report

print(classification_report(y_test, y_pred))

# Confusion Matrix

cm = confusion_matrix(y_test, y_pred)

print(cm)
```

```
The Decision Tree model achieved an accuracy of 89.58%.
              precision    recall  f1-score   support

           0       0.92      0.97      0.94      7952
           1       0.62      0.35      0.45      1091

    accuracy                           0.90      9043
   macro avg       0.77      0.66      0.69      9043
weighted avg       0.88      0.90      0.88      9043

[[7721  231]
 [ 711  380]]
```

Visual Confusion Matrix

In [35]:

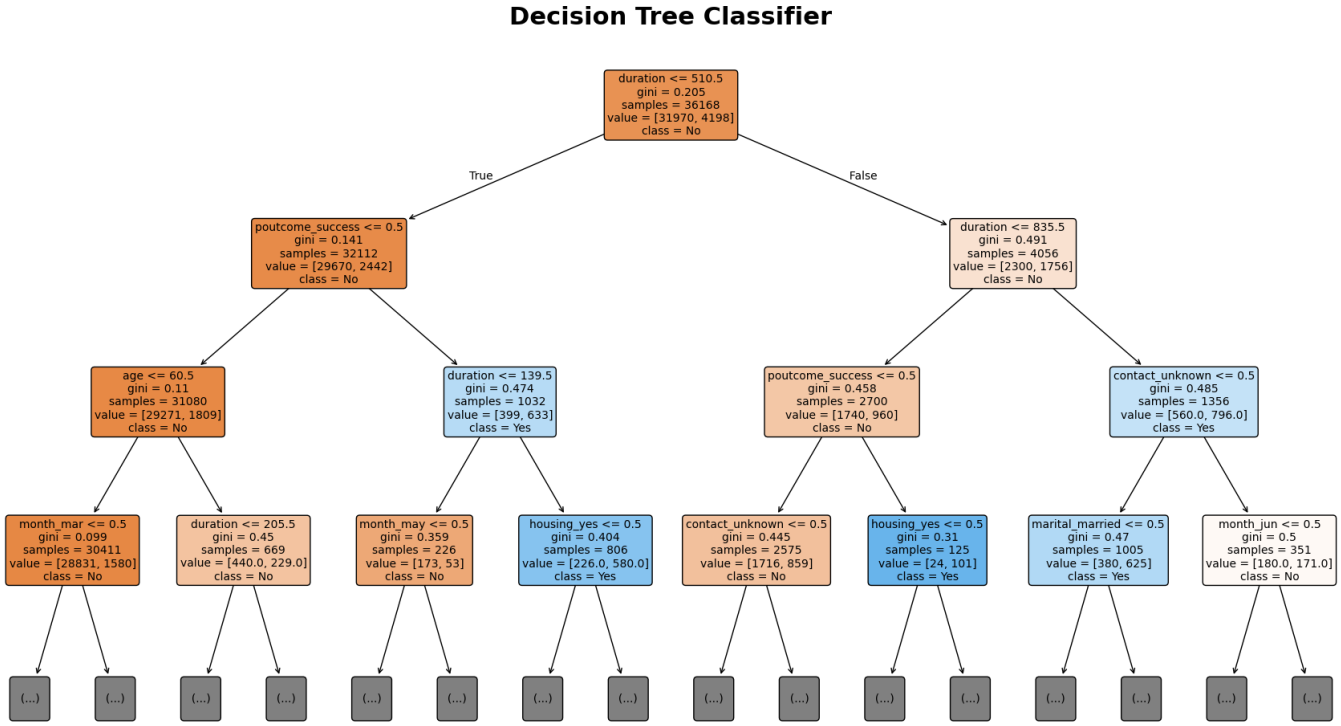
```
plt.figure(figsize=(5, 4))

sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues'
)

plt.title(
    'Confusion Matrix',
    fontsize=12,
    fontweight='bold'
)

plt.xlabel('Predicted')
plt.ylabel('Actual')

plt.tight_layout()
plt.show()
```



Decision Tree Visualization

In [36]:

```
# Import Plot Tree

from sklearn.tree import plot_tree

# Visualize Decision Tree

plt.figure(figsize=(22,12))

plot_tree(
    dt_model,
    feature_names=X.columns,
    class_names=['No', 'Yes'],
    filled=True,
    rounded=True,
    fontsize=10,
    max_depth=3
)

plt.title(
    'Decision Tree Classifier',
    fontsize=22,
    fontweight='bold'
)

plt.show()
```



Feature Importance

In [37]:

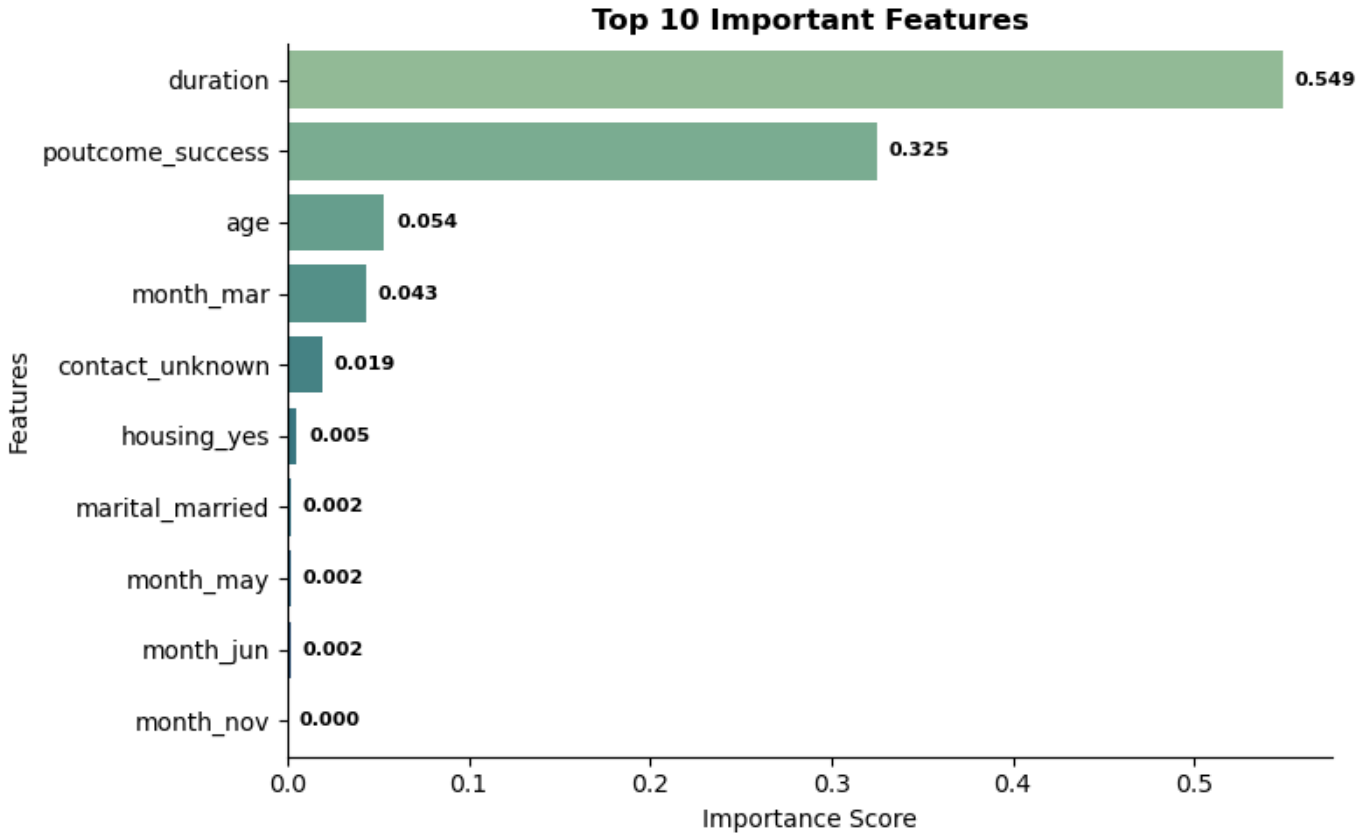
```
importance = pd.DataFrame({
    'Feature': X.columns,
    'Importance': dt_model.feature_importances_
})

importance = importance.sort_values(
    by='Importance',
    ascending=False
).head(10)

print(importance)
```

```
             Feature  Importance
3           duration    0.548931
40  poutcome_success    0.324850
0                age    0.053562
34         month_mar    0.043113
27   contact_unknown    0.019062
24       housing_yes    0.005001
18   marital_married    0.001943
35         month_may    0.001776
33         month_jun    0.001762
36         month_nov    0.000000
```

In [38]:

```
plt.figure(figsize=(8,5))

ax = sns.barplot(
    data=importance,
    x='Importance',
```

```
    y='Feature',
    hue='Feature',
    palette='crest',
    legend=False
)

# Labels
for p in ax.patches:

    width = p.get_width()

    ax.annotate(
        f'{width:.3f}',

        (
            width,
            p.get_y() + p.get_height()/2
        ),

        ha='left',
        va='center',

        fontsize=8,
        fontweight='bold',

        xytext=(5, 0),
        textcoords='offset points'
    )

sns.despine(top=True, right=True)

plt.title(
    'Top 10 Important Features',
    fontsize=12,
    fontweight='bold'
)

plt.xlabel('Importance Score')
plt.ylabel('Features')

plt.tight_layout()
plt.show()
```



Conclusion / Key Insights

→ A Decision Tree Classifier was built to predict whether a customer would subscribe to a term deposit.

→ Data preprocessing included encoding categorical variables and splitting the dataset into training and testing sets.

→ The model achieved good classification performance on unseen data.

→ Features such as contact duration, previous campaign outcome, and month of contact were among the most influential predictors.

→ Visualization of the decision tree helped interpret the model's decision-making process.

→ Feature importance analysis highlighted the factors most affecting customer subscription behavio